losing the game. Hence, the reward signal in these games is sparse, making it difficult for trial-and-error based learning to reliably learn anything, as it requires seeing a reward fairly frequently.

In Gridworld, we've designed the game so that any move that doesn't win the game receives a reward of –1, the winning move gets a reward of +10, and a losing move rewards –10. It's really only the final move of the game where the algorithm can say "Aha! Now I get it!" Since each episode of a Gridworld game can be won in a fairly small number of moves, the sparse reward problem isn't too bad, but in other games it is such a significant problem that even the most advanced reinforcement learning algorithms have yet to reach human-level performance. One proposed method of dealing with this is to stop relying on the objective of maximizing expected rewards and instead instruct the algorithm to seek novelty, through which it will learn about its environment, which is something we'll cover in chapter 8.

3.2.5. Building the network

Let's dig into how we will build our deep learning algorithm for this game. Recall that a neural network has a particular kind of architecture or network topology. When you build a neural network, you have to decide how many layers it should have, how many parameters each layer has (the "width" of the layer), and how the layers are connected. Gridworld is simple enough that we don't need to build anything fancy. We can get away with a fairly straightforward feedforward neural network with only a few layers, using the typical rectified linear activation unit (ReLU). The only parts that require some more careful thought are how we will represent our input data, and how we will represent the output layer.

We'll cover the output layer first. In our discussion of Q-learning, we said that the Q function is a function that takes some state and some action and computes the value of that state-action pair, $Q(s,a)$. This is how the Q function was originally defined (figure 3.5). As we noted in the previous chapter, there is also a state-value function, usually denoted $V_{\pi}(s)$, that